In [1]:

```python
import math
import random
from collections import defaultdict, Counter
import nltk
from nltk.corpus import treebank

nltk.download('treebank', quiet=True)


class HMMTagger:
    """
    First-order Hidden Markov Model for POS tagging.

    States  : POS tags
    Observations : words
    Parameters:
        A[t_i-1][t_i]  = P(t_i | t_i-1)   transition matrix
        B[t][w]        = P(w | t)           emission matrix
    Both matrices are stored as log-probabilities with Laplace smoothing.
    """

    def __init__(self):
        self.transition = defaultdict(lambda: defaultdict(float))
        self.emission = defaultdict(lambda: defaultdict(float))
        self.tag_counts = Counter()
        self.vocab = set()
        self.tags = []

    # -------------------------
    # TRAINING
    # -------------------------

    def train(self, tagged_sentences: list):
        """
        Build transition and emission matrices from tagged sentences.

        Each sentence is a list of (word, tag) tuples.
        Applies Laplace (add-1) smoothing to both matrices to handle
        unseen transitions and emissions at decode time.

        Args:
            tagged_sentences: list of list of (word, tag) tuples
        """
        transition_counts = defaultdict(Counter)
        emission_counts = defaultdict(Counter)

        for sentence in tagged_sentences:
            prev_tag = "<START>"
            for word, tag in sentence:
                self.vocab.add(word)
                self.tag_counts[tag] += 1
                emission_counts[tag][word] += 1
                transition_counts[prev_tag][tag] += 1
```

```
            prev_tag = tag
        transition_counts[prev_tag]["<END>"] += 1

    self.tags = list(self.tag_counts.keys())

    # Laplace smoothing: P(t_j | t_i) = (count(t_i, t_j) + 1) / (count(t_i
    for prev_tag in transition_counts:
        total = sum(transition_counts[prev_tag].values())
        V = len(self.tags) + 1  # +1 for <END>
        for tag in self.tags + ["<END>"]:
            count = transition_counts[prev_tag][tag]
            self.transition[prev_tag][tag] = math.log((count + 1) / (total

    # Laplace smoothing: P(w | t) = (count(t, w) + 1) / (count(t) + |V|)
    for tag in emission_counts:
        total = sum(emission_counts[tag].values())
        V = len(self.vocab)
        for word in self.vocab:
            count = emission_counts[tag][word]
            self.emission[tag][word] = math.log((count + 1) / (total + V))


def _handle_oov(self, word: str) -> str | None:
    """
    Suffix heuristic for out-of-vocabulary words.
    Maps morphological patterns to likely Penn Treebank POS tags.

    Note: task spec uses universal tags (VERB, ADV, NOUN, PROPN) but
    Penn Treebank uses its own tagset, so we map to PTB equivalents.
    """
    if word.endswith("ing"):
        return "VBG"      # VERB (gerund/present participle)
    elif word.endswith("ly"):
        return "RB"       # ADV (adverb)
    elif word.endswith("ness") or word.endswith("tion"):
        return "NN"       # NOUN
    elif word[0].isupper():
        return "NNP"      # PROPN (proper noun)
    return None

def _get_emission(self, tag: str, word: str) -> float:
    """
    Return log P(word | tag).
    For known words, look up the smoothed emission table.
    For OOV words, use the suffix heuristic: assign log(1.0) if the
    heuristic matches the tag, otherwise a small penalty log(1e-6).
    """
    if word in self.vocab:
        return self.emission[tag].get(word, float('-inf'))
    heuristic_tag = self._handle_oov(word)
    return math.log(1.0) if heuristic_tag == tag else math.log(1e-6)
```

```python
    def predict(self, sentence: list[str]) -> list[str]:
        """
        Run the Viterbi algorithm and return the most probable tag sequence.

        Viterbi is exact dynamic programming over the HMM lattice:
          - V[t][tag] = log P of the best path ending in `tag` at position t
          - backpointer[t][tag] = the best previous tag at t-1

        This is a GLOBAL sequence-level decision: we do not greedily pick the
        best tag at each step — we find the sequence with maximum joint probab
        (See Phase IV Q3 for the parallel with Transformer attention.)

        Args:
            sentence: list of word strings

        Returns:
            list of POS tag strings, same length as sentence
        """
        V = [{}]
        backpointer = [{}]

        # Initialisation step: t = 0
        for tag in self.tags:
            trans_prob = self.transition["<START>"].get(tag, float('-inf'))
            emit_prob = self._get_emission(tag, sentence[0])
            V[0][tag] = trans_prob + emit_prob
            backpointer[0][tag] = None

        # Recursion step: t = 1 ... T-1
        for t in range(1, len(sentence)):
            V.append({})
            backpointer.append({})
            for curr_tag in self.tags:
                max_prob, best_prev = float('-inf'), None
                for prev_tag in self.tags:
                    prob = (
                        V[t - 1][prev_tag]
                        + self.transition[prev_tag].get(curr_tag, float('-inf'
                        + self._get_emission(curr_tag, sentence[t])
                    )
                    if prob > max_prob:
                        max_prob, best_prev = prob, prev_tag
                V[t][curr_tag] = max_prob
                backpointer[t][curr_tag] = best_prev

        # Termination: find best final tag accounting for <END> transition
        max_prob, best_last_tag = float('-inf'), None
        for tag in self.tags:
            prob = V[-1][tag] + self.transition[tag].get("<END>", float('-inf'
            if prob > max_prob:
                max_prob, best_last_tag = prob, tag

        # Backtracking: recover full tag sequence
```

```
        best_path = [best_last_tag]
        for t in range(len(sentence) - 1, 0, -1):
            best_path.insert(0, backpointer[t][best_path[0]])

        return best_path



    def evaluate(self, test_data: list) -> float:
        """
        Token-level accuracy on held-out tagged sentences.

        Args:
            test_data: list of list of (word, tag) tuples

        Returns:
            Accuracy as a float in [0, 1]
        """
        correct, total = 0, 0
        for sentence in test_data:
            words = [w for w, _ in sentence]
            true_tags = [t for _, t in sentence]
            pred_tags = self.predict(words)
            for true, pred in zip(true_tags, pred_tags):
                correct += (true == pred)
                total += 1
        return correct / total if total > 0 else 0.0




if __name__ == "__main__":
    sentences = list(treebank.tagged_sents())
    random.shuffle(sentences)

    split = int(0.8 * len(sentences))
    train_data = sentences[:split]
    test_data = sentences[split:]

    print(f"Train sentences: {len(train_data)} | Test sentences: {len(test_dat

    tagger = HMMTagger()
    tagger.train(train_data)
    print(f"Vocabulary size: {len(tagger.vocab)} | Tags: {len(tagger.tags)}")

    accuracy = tagger.evaluate(test_data)
    print(f"Token-level Accuracy: {accuracy:.4f}")

    # Sample prediction
    sentence = ["The", "quick", "brown", "fox", "is", "running", "quickly"]
    tags = tagger.predict(sentence)
    print("\nSample prediction:")
    for word, tag in zip(sentence, tags):
```

```
        print(f"  {word:15} {tag}")
```

```
Train sentences: 3131 | Test sentences: 783
Vocabulary size: 11034 | Tags: 46
Token-level Accuracy: 0.8688

Sample prediction:
  The             DT
  quick           JJ
  brown           NN
  fox             NN
  is              VBZ
  running         -NONE-
  quickly         .
```

In [ ]:

```
"""
hmm_tagger.py — Phase II: HMM POS Tagger with Viterbi Decoding
Cryptonite Research Training | NLP Specialisation Task 2

Trains a first-order Hidden Markov Model on Penn Treebank (via NLTK),
implements the Viterbi algorithm for decoding, and handles OOV words
via suffix heuristics.

PHASE IV — ERROR ANALYSIS (inline)
====================================

Q1: 3 sentences where HMM fails but BERT succeeds (Markov assumption):

  1. "The bank can guarantee deposits will eventually cover future tuition cos
     HMM tags "bank" as NN (most frequent) but misses financial context built
     from the full sentence. BERT attends to "deposits", "tuition", "costs"
     bidirectionally and correctly resolves the sense.

  2. "The horse raced past the barn fell"
     Garden-path sentence. HMM greedily commits to "raced" as VBD too early
     because its Markov window can't backtrack over the structural ambiguity.
     BERT's attention over the whole sequence handles the reduced relative cla

  3. "I saw the man with the telescope"
     PP attachment ambiguity. HMM uses only local bigram context to tag "with"
     and the surrounding words. BERT uses sentence-level context to better
     resolve whether "with" attaches to "man" or "saw".

  Root cause: the Markov assumption P(ti | ti-1) means the model has no memory
  beyond one tag. Any dependency spanning >1 tag is invisible to the HMM.

Q2: 1 sentence where HMM is correct but BERT is wrong:

  "IBM shares rose yesterday."
  HMM correctly tags "IBM" as NNP because it appears in training with that tag
  BERT (bert-mini, fine-tuned on limited CoNLL data) occasionally mis-tags
  short, syntactically simple sentences with well-known proper nouns because
  the small model underfit proper noun patterns. This suggests that on clean,
  formulaic financial/news sentences, a count-based model with good coverage
```

```
  can outperform a small neural model trained on insufficient data.

Q3: Viterbi vs Attention — the parallel:

  Viterbi is a dynamic programming algorithm that finds the globally most
  probable tag sequence by maintaining, at each step, the best path to every
  possible state. It is a global sequence-level inference mechanism: the final
  decision at position t is informed by all previous positions via the DP tabl

  Transformer attention is also a global mechanism: every token at position t
  attends to every other token in the sequence simultaneously, computing a
  weighted combination of all positions. Both avoid greedy local decisions
  in favour of considering the full sequence context.

  The key difference: Viterbi is exact inference over a fixed probabilistic
  graphical model (the HMM). Attention is a learned, soft, differentiable
  weighting that does not correspond to a single probabilistic model — it is
  one component of a larger learned function. Viterbi looks backward through
  a DP lattice; attention looks in all directions simultaneously.
"""
```